

Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 23

Search algorithms and Introduction to Graphs



Universit 
Bocconi
MILANO

Objectives of the lesson

- Understand how search algorithms work
- Introduce graphs
- Understand what is a tree
- Understand what are Shortest-path algorithms
- Understand how Dijkstra's shortest-path algorithm works

Search algorithms

The goal of a **search algorithm** is to efficiently locate a particular item (or group of items) from a set, such as an element in a list, a file in a directory, or the best path between two points in a network

The two most common and simple examples of search algorithms are:

- ✓ Linear Search
- ✓ Binary search

Linear search

- Linear Search (also known as Sequential Search) goes through each element in a list **one by one** from start to end, comparing each element with the target value
- As it finds a match, it returns the related output (e.g. its index), otherwise it continues until the end of the list
- Main **advantages**: it works on any list (sorted or unsorted) and it is easy to implement
- Main **disadvantages**: it is not very efficient (too slow for large datasets)

Binary search

- Binary Search repeatedly **divides in half** a sorted list of elements to find the target value
- It starts by checking the middle element. If the target value is lower, it checks the middle element of the left half, if it is greater, it checks the middle element of the right half. The procedure is repeated until the target value is found
- Main **advantages**: it is very efficient for large sorted datasets (low time complexity)
- Main **disadvantages**: it only works on sorted lists and it is more complex to implement than the linear search

Search example

Given (as **input**) the following sorted list of 16 elements (i.e. $n = 16$) we want to obtain (as **output**) the index corresponding to the value **47**, using both the **linear search** and **binary search** approaches

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90



Linear search

Is it **47** equal to **-25** ?

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90



...No !



Linear search

Is it **47** equal to **-19** ?

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90



...No !



Linear search

Is it **47** equal to **-10** ?

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90



...No !



Linear search

Is it **47** equal to **-5** ?



...No !



Linear search

Is it **47** equal to **9** ?



...No !



Linear search

Is it **47** equal to **11** ?



...No !



Linear search

Is it **47** equal to **25** ?

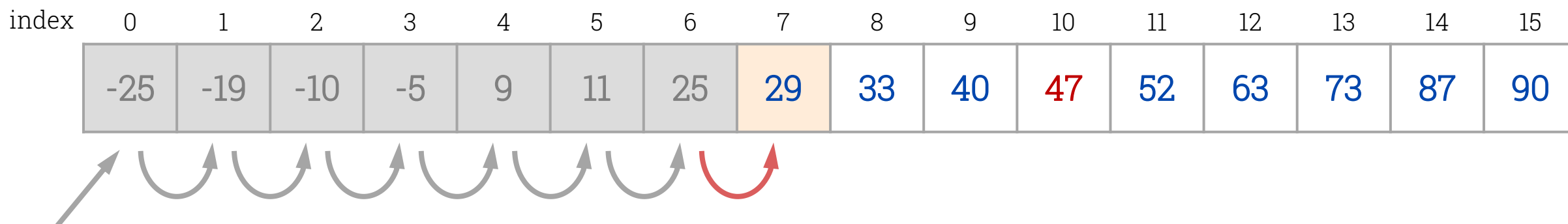
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90

...No !



Linear search

Is it **47** equal to **29** ?

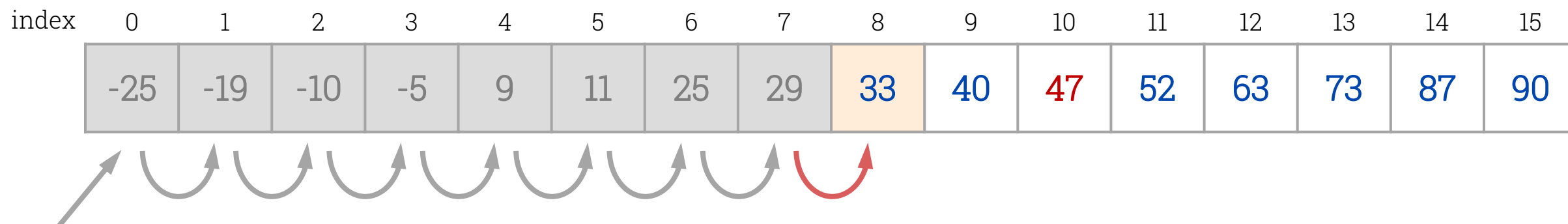


...No !



Linear search

Is it **47** equal to **33** ?

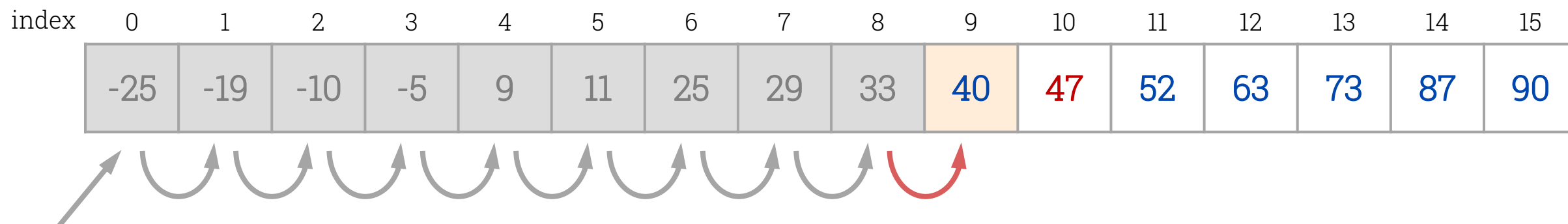


...No !



Linear search

Is it **47** equal to **40** ?

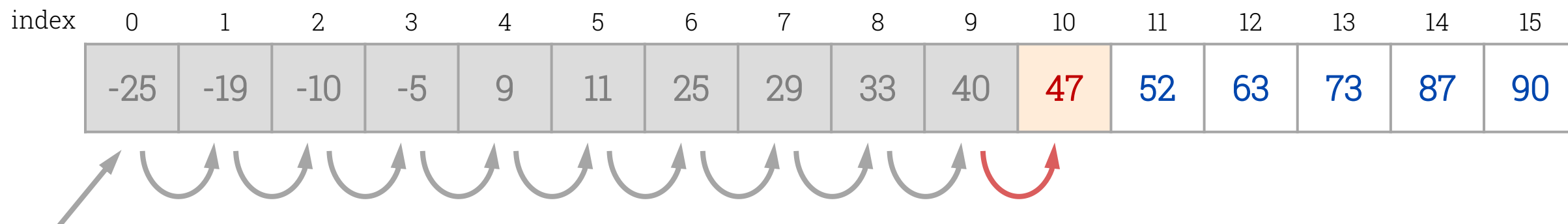


...No !



Linear search

Is it **47** equal to **47** ?

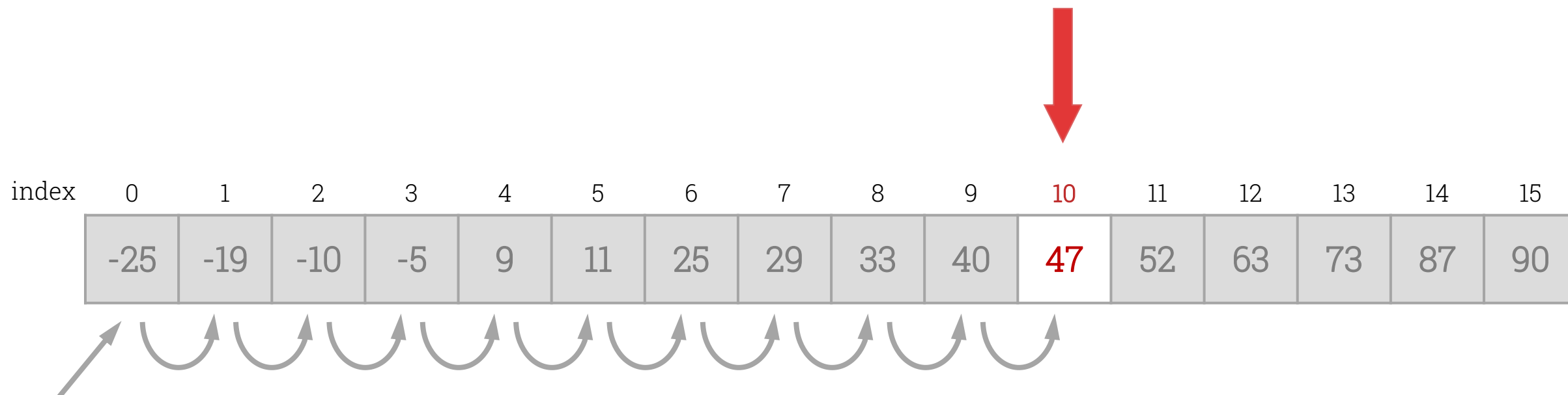


...Yes !



Linear search

It means that the index of **47** is 10 !



Linear search

- With linear search, it took 11 steps to obtain the index of value **47**
- Using the same approach, it would take 16 steps (i.e. **n**) to obtain the index of value **90**
- In other words, the **complexity** of the linear search algorithm **grows proportionally** to the input:

$$O(n)$$

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90



Binary search

Let's now see how many steps it takes to find the index of value **47** in the same sorted list, using **binary search**:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90



Binary search

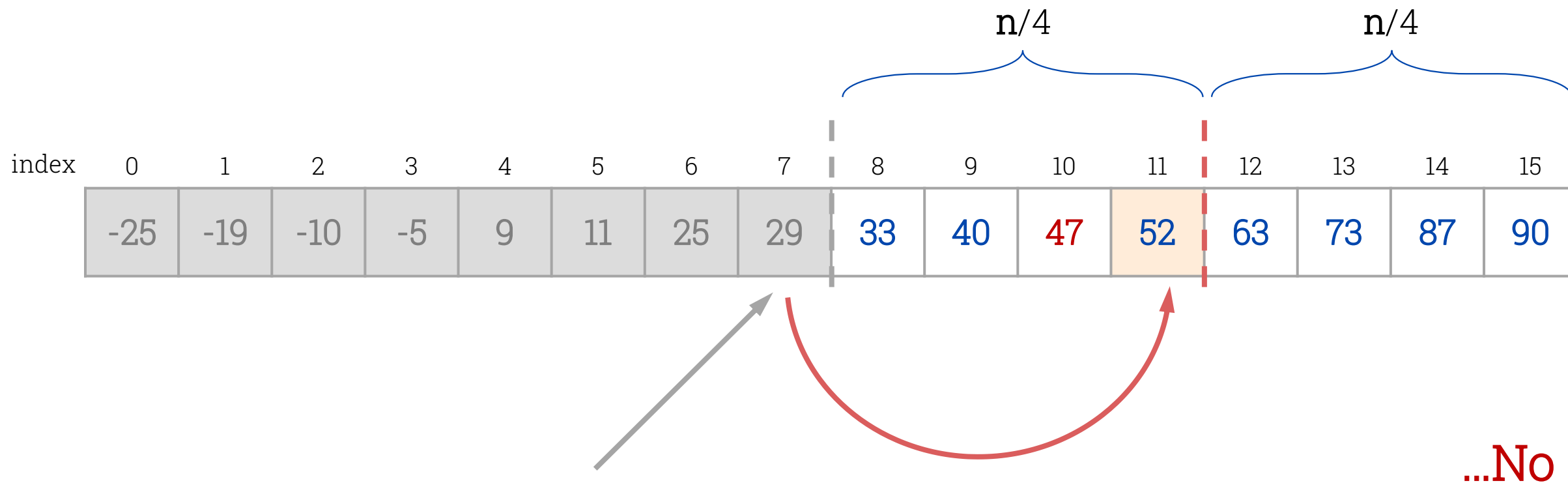
Is it **47** greater than **29** ?

	$n/2$								$n/2$							
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90

...Yes!

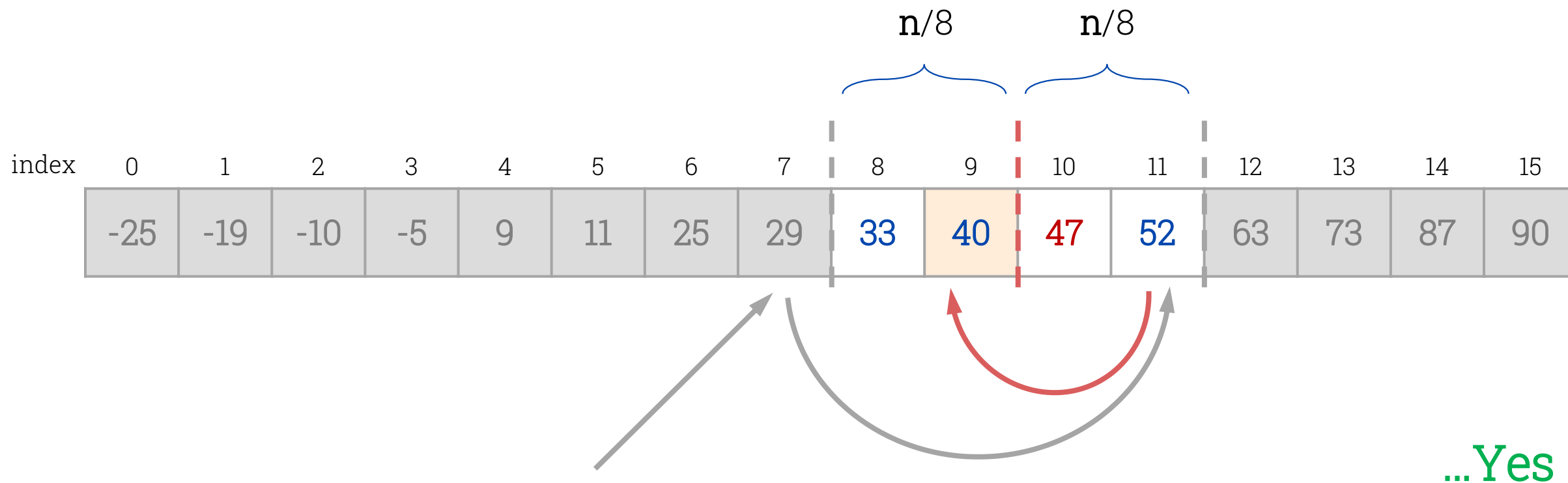
Binary search

Is it **47** greater than **52** ?



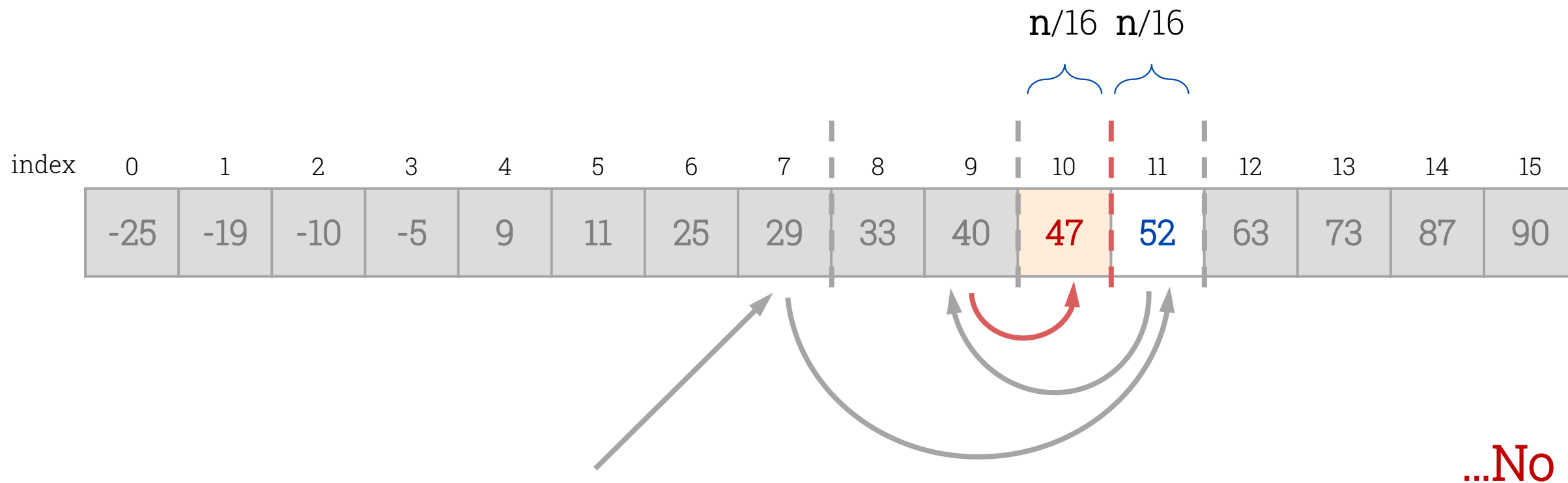
Binary search

Is it **47** greater than **40** ?



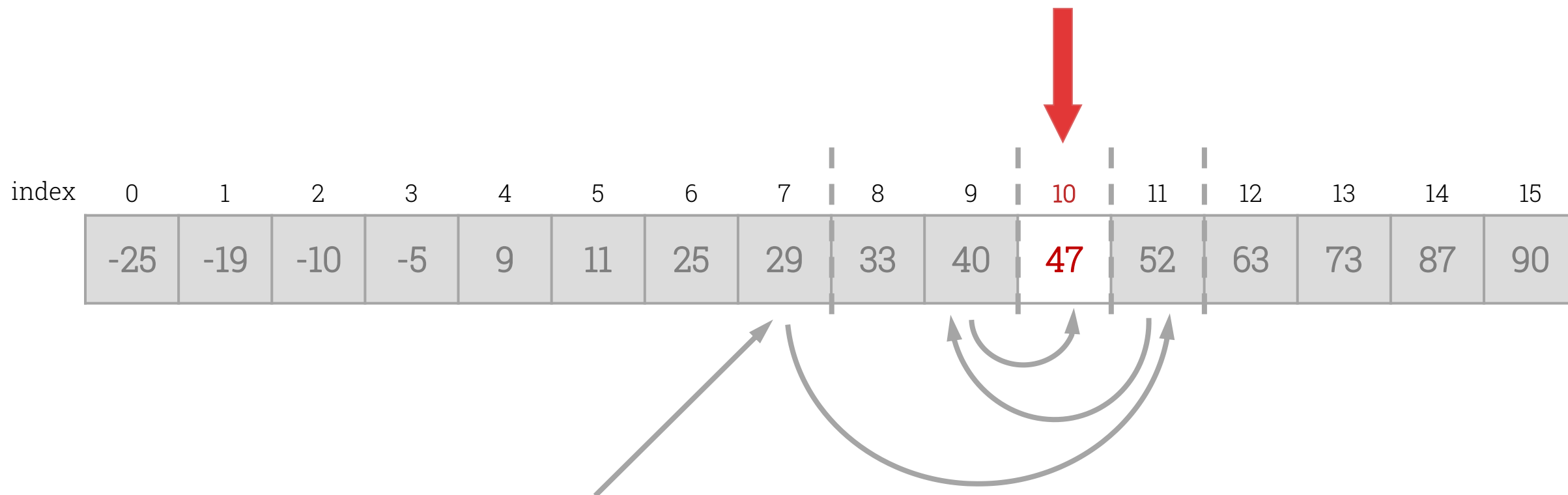
Binary search

Is it **47** greater than **47** ?



Binary search

It means that the index of **47** is 10 !



Binary search

- With binary search, it took 4 steps to obtain the index of value **47**
- Using this approach, no more than 4 steps are needed to obtain the index of any value in a list of $n = 16$ elements (where $16 = 2^4$ and $4 = \log_2 n$)
- In other words, the **complexity** of the binary search algorithm **grows more slowly** than the input:

$$O(\log_2 n)$$

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	-25	-19	-10	-5	9	11	25	29	33	40	47	52	63	73	87	90



Evidences

To evaluate the **efficiency** of various approaches (available in Python) to obtain the index of an element in a sorted list, we can observe the outputs of “**Word index.py**” script

The program uses the collection of **234,377 words** available in the file “**MyWords.txt**”, returning the index (and the running time needed to find it) of the word enter by the user at the call of the function **test()**

```
>>> test()
Enter the word of which you want to know the position: zoom
-----
index   | milliseconds | Approach used
-----
234035  | 1.646000077  | .index method of lists
234035  | 4.097400000  | Traversing elements in a list
234035  | 4.726000014  | Indexing with a "for" loop
234035  | 12.216900010 | Indexing with a "while" loop
234035  | 0.006199931  | Binary search
-----
```



Exercise

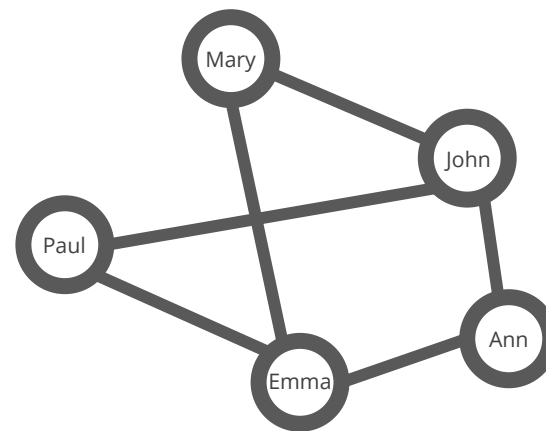
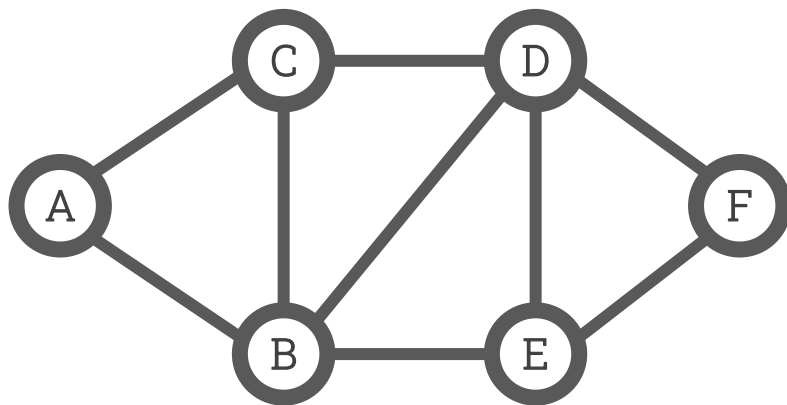
- Create a program to guess the number the user is thinking of
- The program must ask the user to think of a number (in an assigned range) and must ask as few questions as possible to guess it

Graphs

- A graph is a **non-linear** data structure, that consists of **nodes** and **edges** (connections between nodes)
- Graphs are powerful modeling tools that are used to solve real-world problems
- It is possible to model a particular problem in a graph and then apply some graph algorithms to find the solution

Graphs

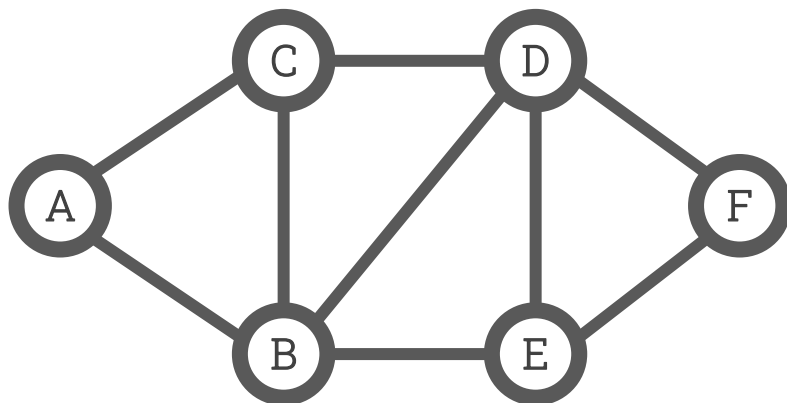
- A very common use of graphs is networks' representation such as computer or telephone networks, city roads and points of interest etc.
- Even in social media, graphs can be used to represent friends or connections (each user is represented as a node and is connected with the nodes that are friends)



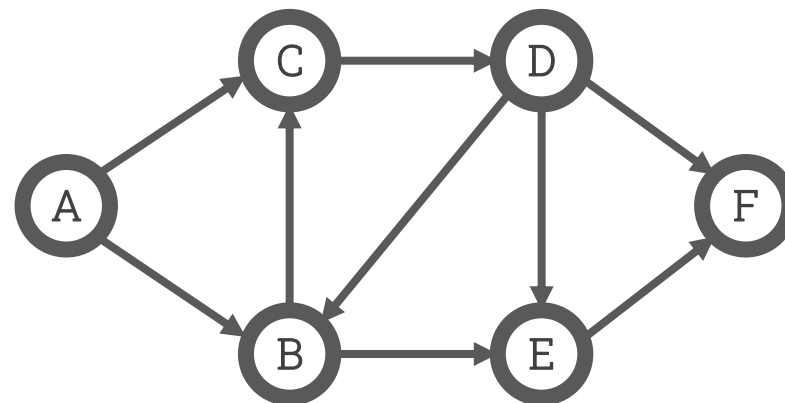
Undirected and directed graphs

- In **undirected graphs**, the movement between two connected vertices is possible in both directions
- In **directed graphs**, the connection between two connected vertices is defined by the direction of the edge

Undirected graph

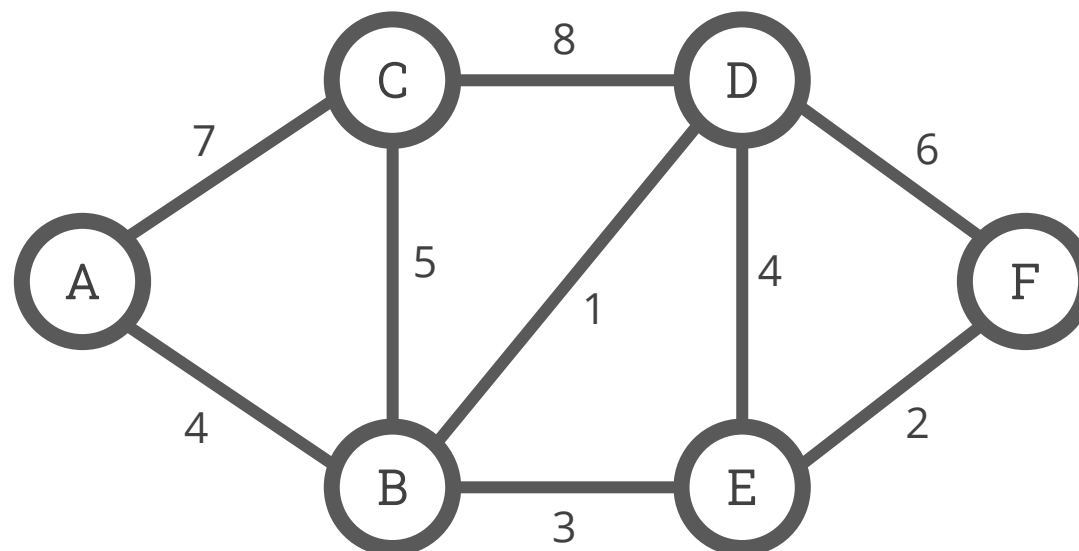


Directed graph



Weights

In most cases, graphs have a **weight value** on each edge, that defines the “cost” of movement between the nodes connected by that particular edge



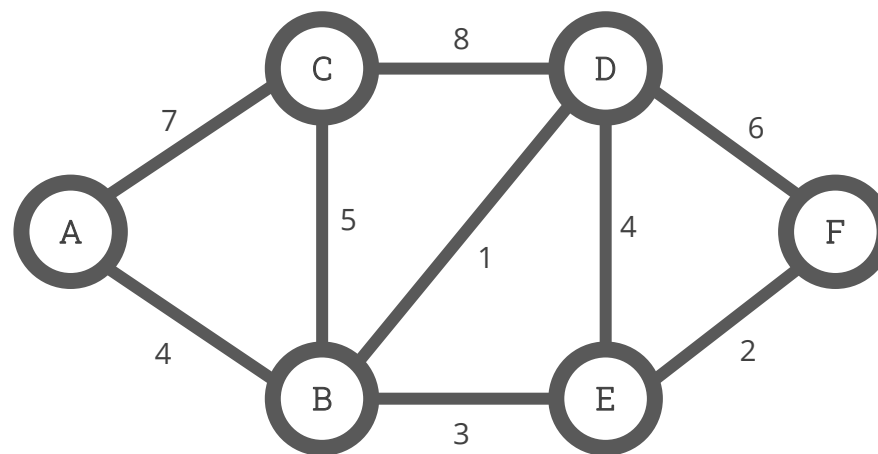
Graph Terminology

- **Vertices:** vertices are nodes of the graph
- **Edges:** edges are used to connect any two vertices
- **Adjacency:** a vertex is adjacent to another vertex if there is an edge connecting them (i.e. vertices A and E are not adjacent because there is no edge between them)
- **Path:** a path is a sequence of edges allowing us to go from a vertex to another (i.e. A-B, B-C and A-C are different paths from vertex A to vertex C)
- **Weight (of an edge):** a numerical value that represents the cost (or distance, time, effort etc.) to connect two vertices in a graph

Vertices and edges

Considering an undirected chart with vertices, edges and the related weights (see example below), we can define collections with a finite number of elements:

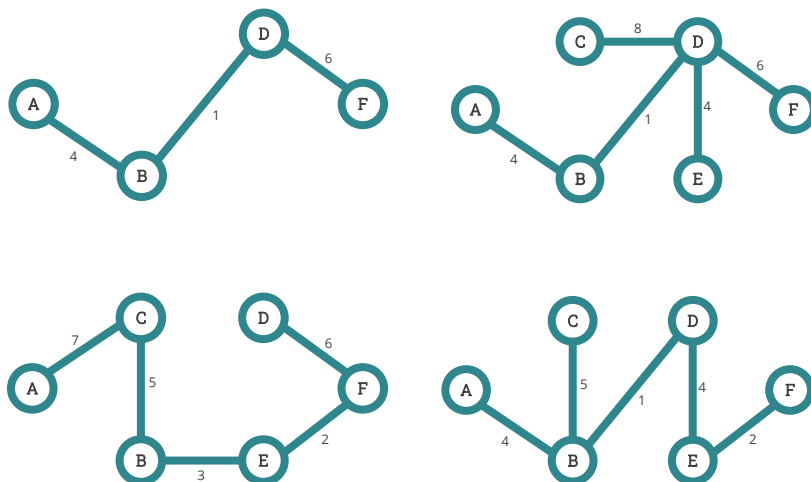
- **Vertices:** A, B, C, D, E, F
- **Edges:** AB, AC, BC, BE, BD, CD, DE, DF, EF
- **Weights:**
AB: 4
AC: 7
BC: 5
BE: 3
BD: 1
CD: 8
DE: 4
DF: 6
EF: 2



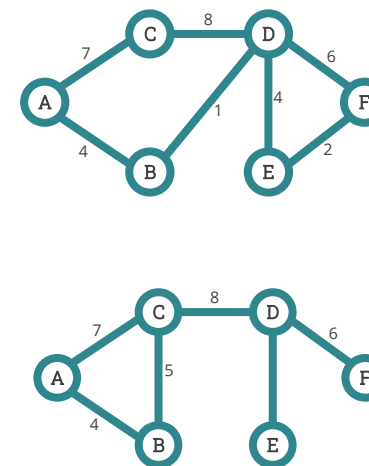
Trees

- A tree is a graph that is **connected** and **acyclic**, meaning that all its vertices are connected at least with another vertex and that it contains no cycles
- A tree is a connected graph with **N** vertices and **N-1** edges

Trees



Not trees

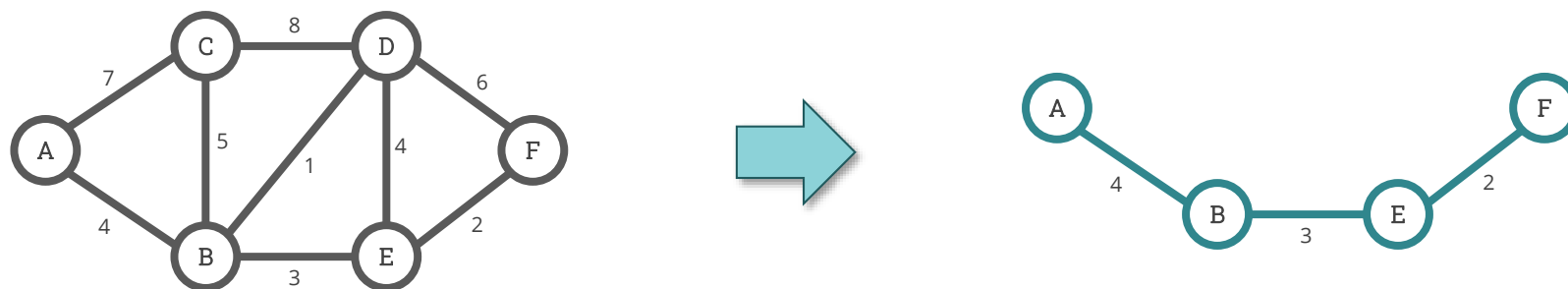


Graph search and optimization algorithms

- Graph **search** and **optimization** problems are solved using different categories of algorithms, among which:
 - shortest-path algorithms
 - spanning tree algorithms
- In shortest-path algorithms we search for path with the minimum cost (weight) between two vertices, without cycles (except specific cases), connecting just the needed vertices of the graph
- In spanning tree algorithms we search for the optimal tree – with the minimum cost (weight) – covering all vertices in the graph

Shortest-path algorithms

- Shortest path algorithms identify the most optimal path between two specific vertices of a graph (with no cycles), based on the weight of the edges, usually representing length or time duration
- The shortest path provides us with detailed navigation instructions to get from one vertex to another vertex in the graph via multiple intermediary vertices if necessary



Dijkstra's algorithm

- One of the most popular shortest path algorithms is the one conceived by Dutch computer scientist Edsger W. Dijkstra in 1956
- Dijkstra's algorithm finds the shortest path (distance) among vertices in a connected graph with non-negative edge weights
- Dijkstra's algorithm can be used both with directed and undirected graphs



Dijkstra's algorithm

Dijkstra's algorithm finds an optimal solution to shortest path problems as long as the graph meets the following assumptions:

- the graph must be **connected**: if the graph is not connected, Dijkstra's algorithm will only find paths to nodes it can reach from the source node
- **weights must be available for all edges**. The algorithm is based on minimizing the total path cost, so without weights it will not be able to find the minimum weight path between two nodes
- all edge weights must be **non-negative**

Dijkstra's algorithm

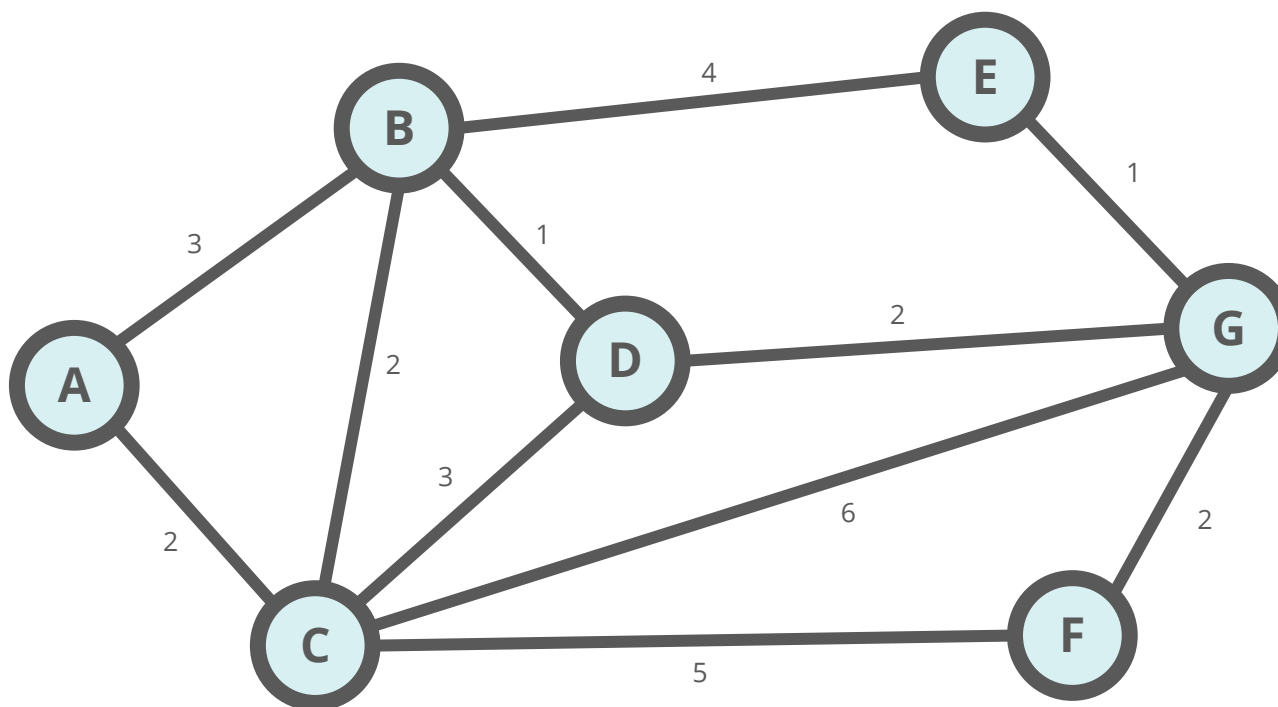
Dijkstra's algorithm starts from a vertex and explores all the vertices in the graph, updating sub-path distances until it finds the shortest sub-path to the destination vertex. The algorithm follows these steps:

1. defines the starting node and sets its distance to 0
2. sets the distance to all other nodes to ∞ (infinity means “unknown distance”) and marks all nodes as “unvisited”
3. selects the neighboring unvisited node with the smallest distance
4. for each neighbor of the selected node, it calculates the tentative distance through the current node and updates it (if lower than its current value)
5. marks the current node as visited and repeats the last steps until all the nodes are visited and the shortest distance to the end node is found
6. as an option, it reconstructs the shortest path keeping track of previous nodes and backtracking from the destination to the starting vertex



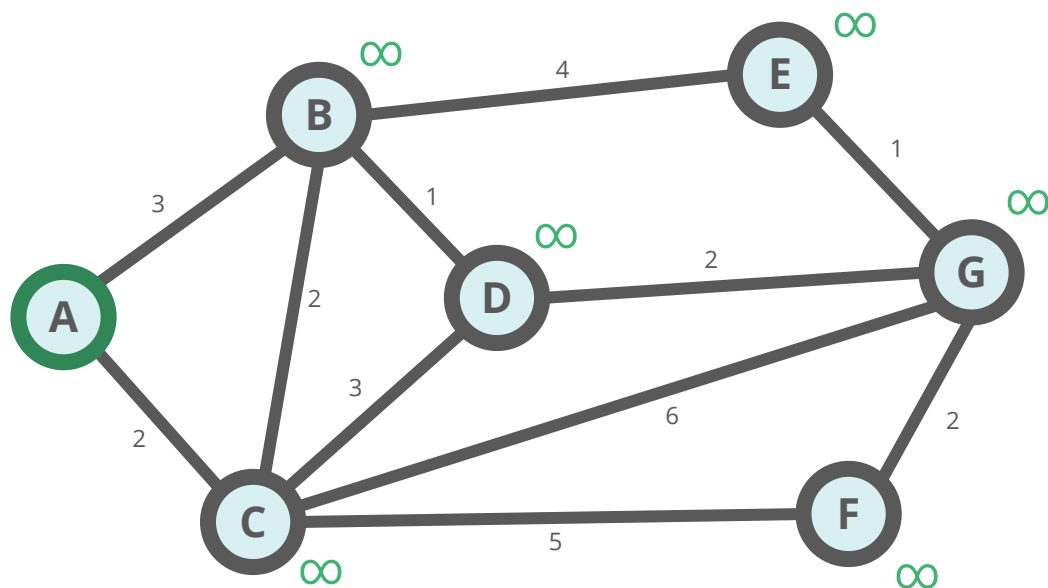
Dijkstra's algorithm example

Consider the following undirected connected graph with weights representing the time in minutes needed to go from one vertex to another. We need to find the shortest (minimum weight) path from vertex A to vertex G using Dijkstra's algorithm



Dijkstra's algorithm example - Step 1

- We want to label each vertex with the shortest time needed to get there starting from vertex A. At first all the nodes are unknown. We start from vertex A (which becomes **explored**) and set its distance to 0 (minutes), then we assign infinity path values to all other vertices

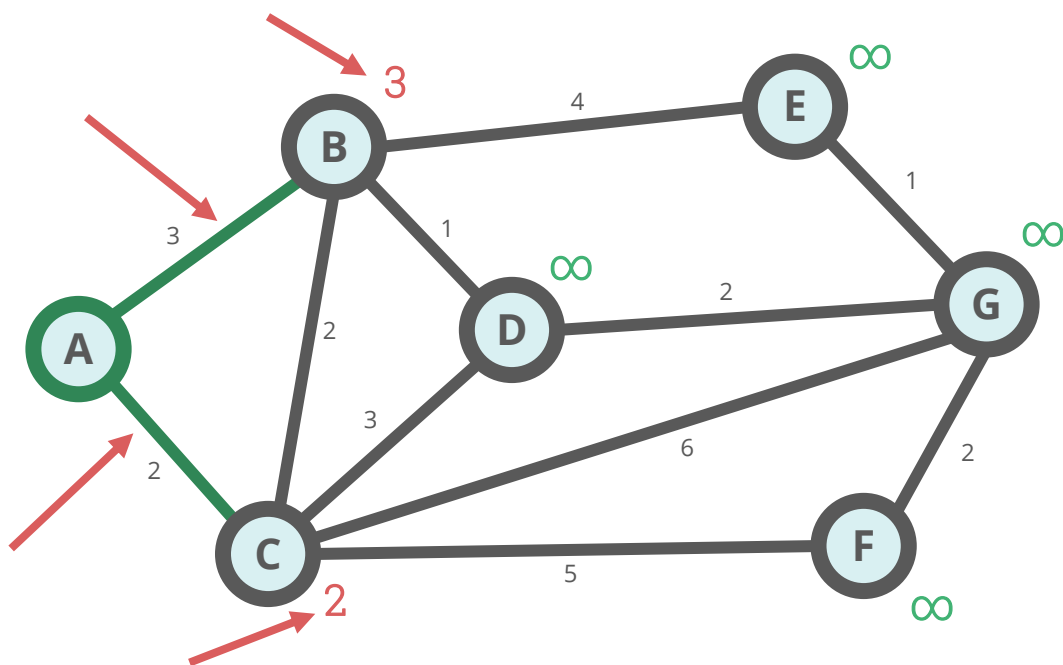


Distances

A: 0
B: ∞
C: ∞
D: ∞
E: ∞
F: ∞
G: ∞

Dijkstra's algorithm example - Step 2

- Starting from A, we look at its neighbors and we can start labeling them with the time needed to reach them from vertex A:
 - from A to C the time needed is 2. We update it since it's much better than infinite
 - from A to B the time needed is 3. We update it since it's much better than infinite



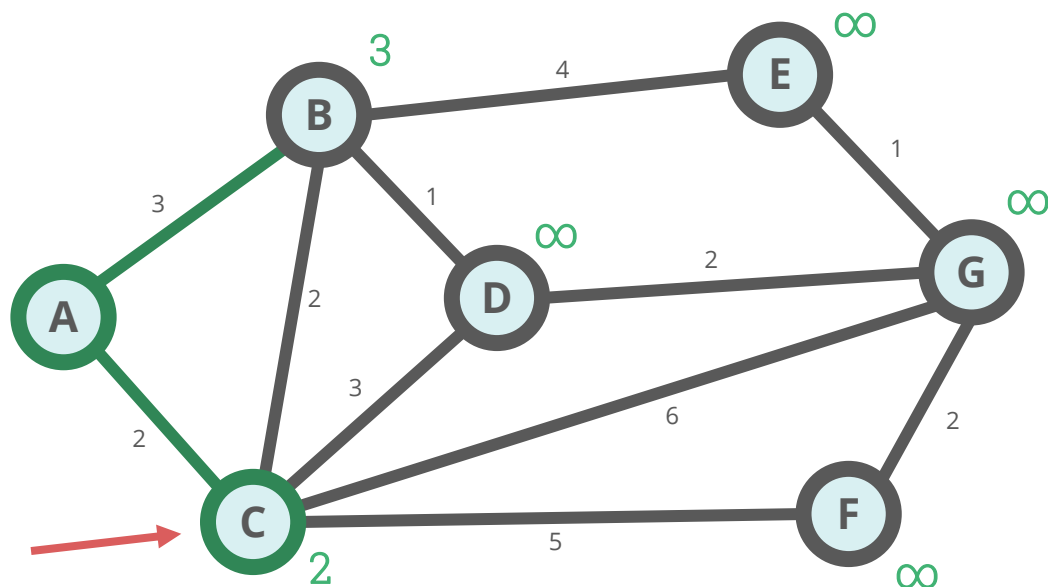
Distances

A: 0
B: 3
C: 2
D: ∞
E: ∞
F: ∞
G: ∞

Dijkstra's algorithm example - Step 3

At each step the algorithm:

- 1) updates the time needed to reach the neighboring vertices
- 2) chooses the next vertex (explores it), picking up the unexplored one with the lowest time needed to be reached from the previous vertex. In this case we explore vertex C

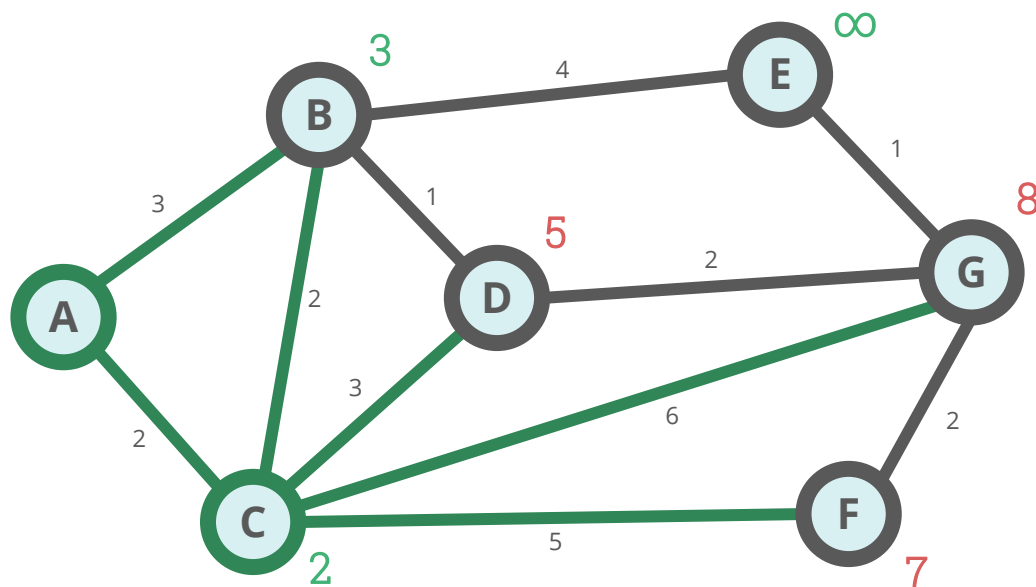


Distances

A: 0
B: 3
C: 2
D: ∞
E: ∞
F: ∞
G: ∞

Dijkstra's algorithm example - Step 3

- The time needed to reach vertex F from vertex C is 5, so the time needed to reach vertex F from vertex A is 7
- The time needed to reach G from C is 6, so from vertex A to G is 8
- The time needed to reach D from C is 3, so from vertex A to D is 5
- The time needed to reach B from C is 2, so from vertex A to B should be 4. But since this value is greater than 3, we don't update the value and we leave 3 (the lowest time we have)

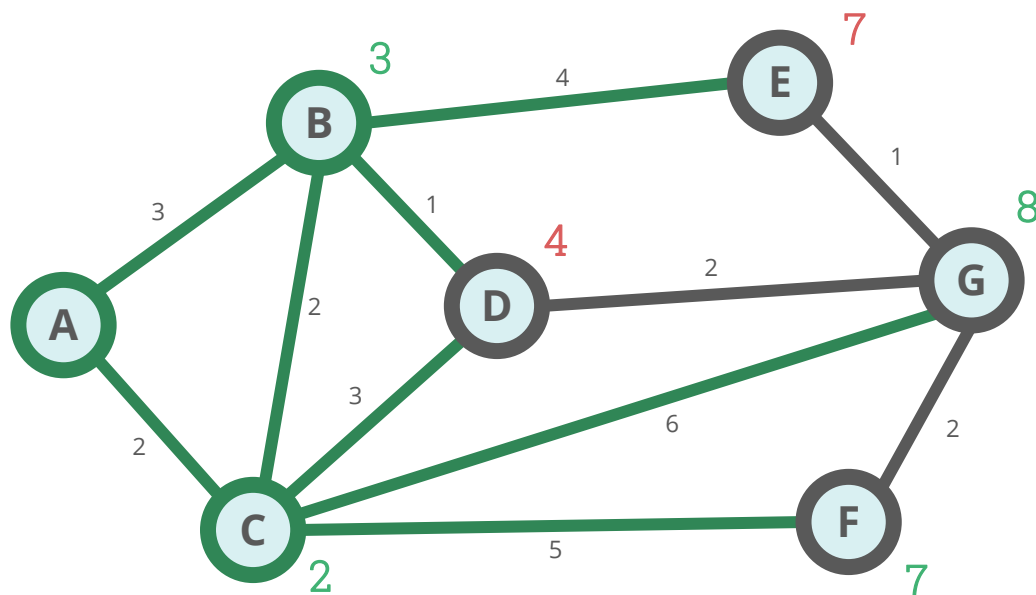


Distances

A: 0
B: 3
C: 2
D: 5
E: ∞
F: 7
G: 8

Dijkstra's algorithm example - Step 4

- Next vertex to be explored is B (the one with the lowest time to reach so far)
- From B we can reach E in 4 minutes, so from vertex A to E is 7
- From B we can reach D in 1 minute, so from vertex A to D is 4. This value is lower than the previous one (5, which was the sum of the edges A-C-D) so we updated it!



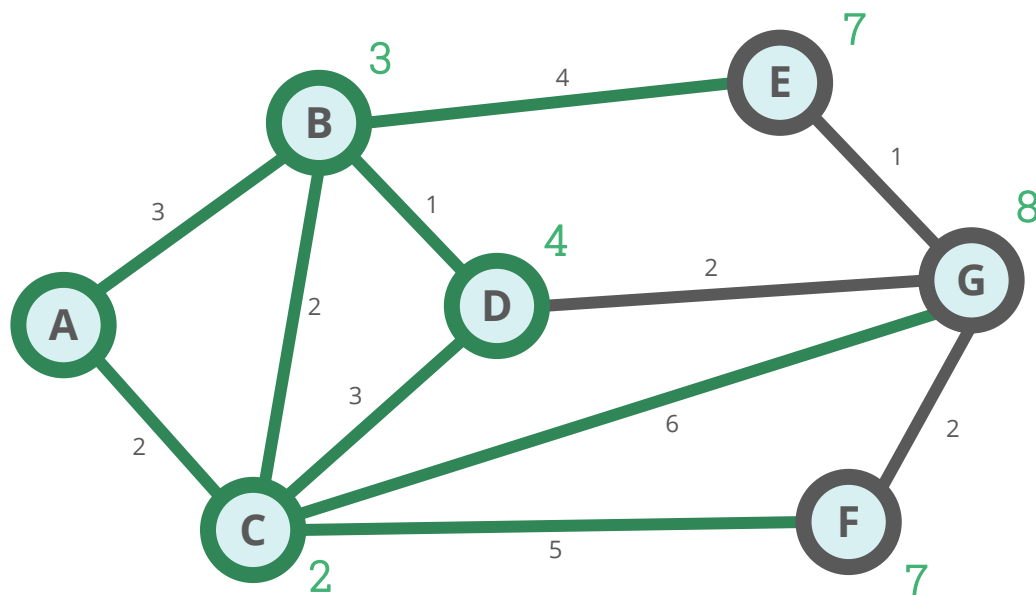
Distances

A: 0
B: 3
C: 2
D: 4
E: 7
F: 7
G: 8

Dijkstra's algorithm example - Step 5

- Next vertex to be explored is D (the one with the lowest time to reach so far).

Note: each time we explore a new vertex it is guaranteed that we have found the shortest path to get from A to that vertex, since if there were some shorter path we would have explored that shorter path first. The algorithm prioritizes exploring smaller values first

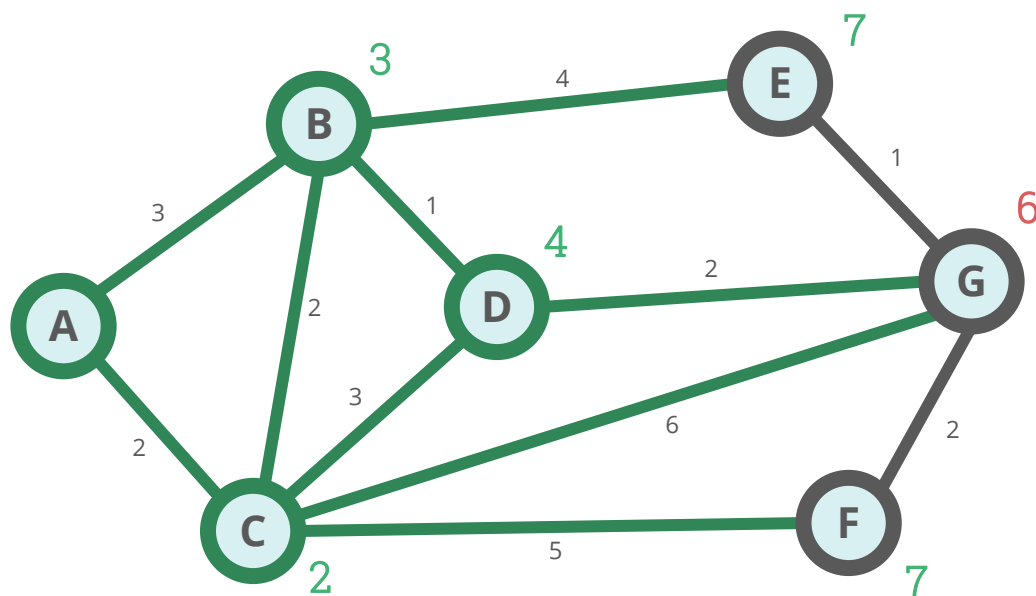


Distances

A: 0
B: 3
C: 2
D: 4
E: 7
F: 7
G: 8

Dijkstra's algorithm example - Step 5

- From D we can reach G in 2 minutes, so from vertex A to G is now 6. Since this value is lower than the previous one, we must update it



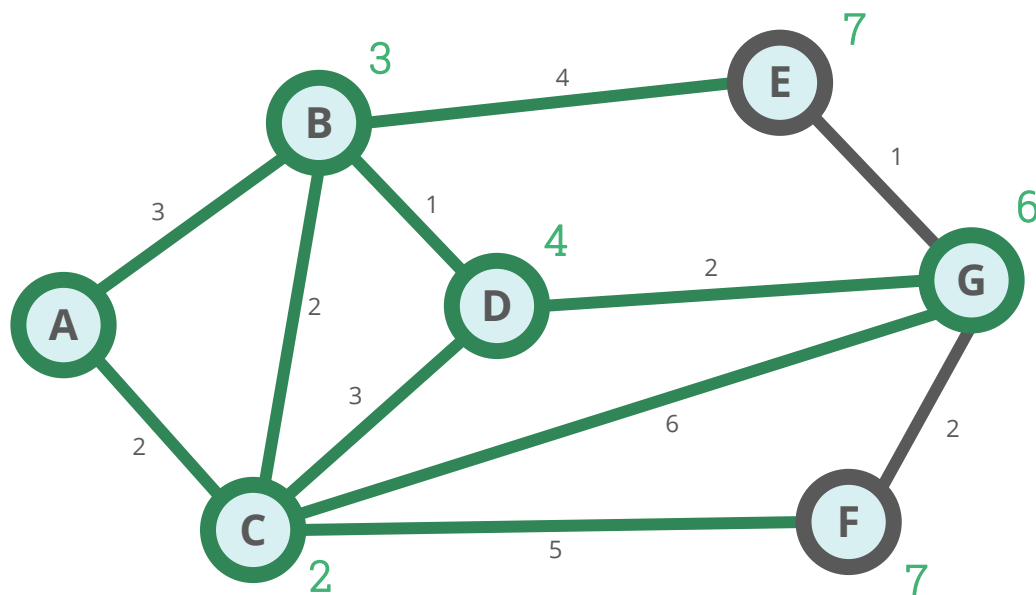
Distances

A: 0
B: 3
C: 2
D: 4
E: 7
F: 7
G: 6



Dijkstra's algorithm example - Step 6

- Next vertex to be explored is G, since it's the unexplored one with the lowest time needed to be reached from the previous vertex (D).
- But G is our destination! This tells us that the shortest time needed to get from A to G is 6 minutes. If a shorter path existed, we would have updated it before exploring this vertex
- The algorithm could continue exploring remaining vertices to find shortest paths from A to all nodes, but since G is our destination, we can already identify the shortest path from A to G as A-B-D-G



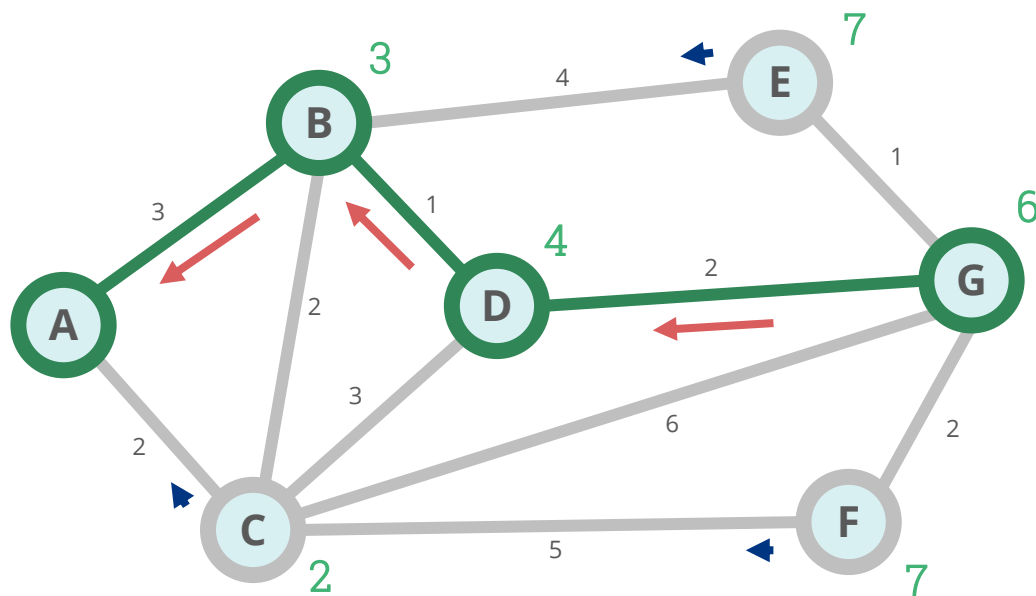
Distances

A: 0
B: 3
C: 2
D: 4
E: 7
F: 7
G: 6

Dijkstra's algorithm example

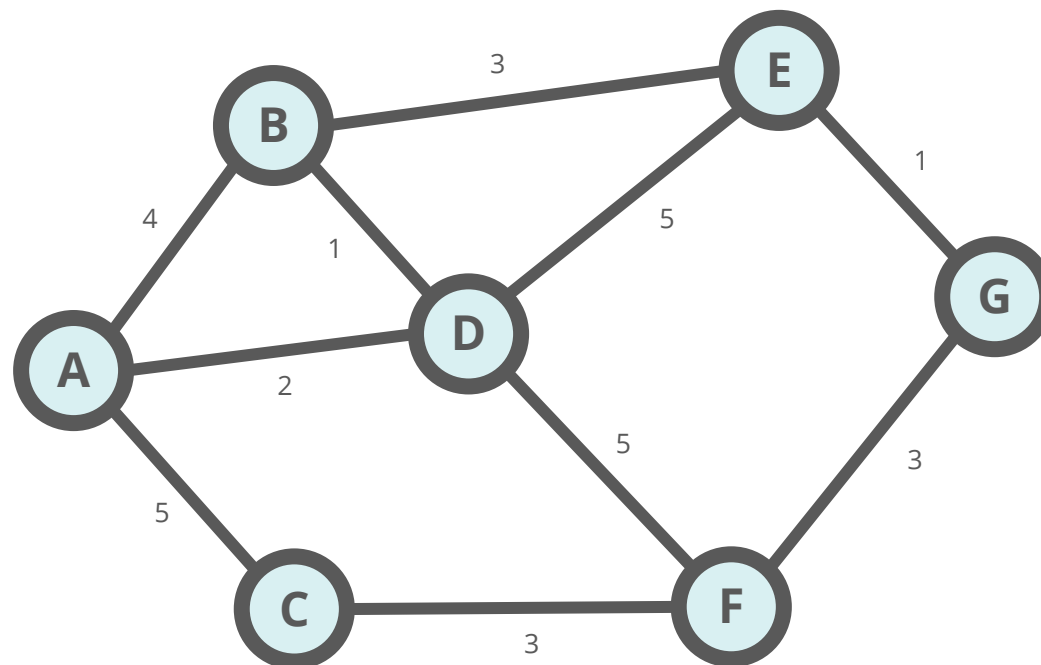
At this point we need to consider that:

- the algorithm has given us how long it will take to get from A to G, but it has not given us which path to follow
- to reconstruct the path we need to add a step to the algorithm so that every time we update the distance needed to reach a vertex, we can keep track of the vertex we came from



Dijkstra's algorithm exercise

Consider the following undirected connected graph with weights representing the time in minutes needed to go from one vertex to another. We need to find the shortest (minimum weight) path from vertex A to vertex G using Dijkstra's algorithm



Topics to learn with today's lecture

- Algorithmic approach to search
- Linear search e binary search
- Graphs
- Shortest-path algorithms
- Dijkstra's shortest-path algorithm

Study materials

This **set of slides** has to be considered the study material for the contents of **Lesson 23**

Any links to external resources have to be considered as supplementary material for further exploration of the topics covered